

1. Practical5b.c

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    pipe(fd);

    if(fork()==0)
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);
    }

    if(fork()==0)
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp("grep", "grep", ".c", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}
```

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if(sig==SIGINT)
        printf("\nSIGINT Received\n");

    if(sig==SIGTERM)
        printf("\nSIGTERM Received\n");

    if(sig==SIGUSR1)
        printf("\nSIGUSR1 Received\n");
}

int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);

    while(1)
    {
        printf("Running...\n");
        sleep(3);
    }

    return 0;
}
```

Ex; kill -SIGUSR1 PID

Ex: kill -SIGTERM PID

```
eesh@DESKTOP-6R05L78:~$ ./po
Running...
Running...
KILLRunning...
kRunning...
ill-Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
^C
SIGINT Received
Running...
Running...
^C
SIGINT Received
Running...
```